École Normale Supérieure de l'Enseignement Technique
Filière : Master –Système distribué et Intelligence Artificielle (SDIA)

Conception, Implémentation et Évaluation Comparative de Métaheuristiques pour l'Optimisation sur Espaces Discrets et Continus

Réalisée par :

Fatima-zahra Massane
Classe : Master – SDIA-1

Encadrée par :

Prof. M. Messtari

Année universitaire 2024 – 2025

Contents

1 Introduction et problématique **2**

2 Rappels théoriques **2**
2.1 Paysage d'optimisation et minima locaux . . . 2
2.2 Descente locale (plus grande pente) . . . 2
2.3 Recherche tabouée (Glover, 1986) . . . 3
2.4 Recuit simulé (Kirkpatrick et al., 1983) . . . 3
2.5 Algorithme génétique et théorème des schèmes (Holland, 1975) . . . 3

3 Modélisation des problèmes **3**
3.1 Volet A – Fonction de coût quadratique binaire . . . 3
3.2 Volet B – Fonction objective continue . . . 5

4 Implémentation des algorithmes **6**
4.1 Architecture du projet . . . 6
4.2 A1 – Descente locale (plus grande pente) . . . 6
4.3 A2 – Recherche tabouée . . . 6
4.4 A3 – Recuit simulé . . . 7
4.5 B2 – Algorithme génétique . . . 8

5 Protocole expérimental **8**

6 Résultats et interprétation **9**
6.1 A1 – Descente locale . . . 9
6.2 A2 – Recherche tabouée . . . 10
6.3 A3 – Recuit simulé . . . 11
6.4 B2 – Opérateurs génétiques . . . 12
6.5 B3 – Schèmes et théorème de Holland . . . 13

7 Comparaison générale **14**
7.1 Tableau synthétique – Meilleures configurations . . . 14
7.2 Discussion scientifique . . . 15

8 Conclusion et perspectives **16**
8.1 Bilan scientifique . . . 16
8.2 Perspectives . . . 16

1 Introduction et problématique

L'optimisation est une discipline fondamentale des mathématiques appliquées et de l'informatique. Elle vise à trouver, parmi l'ensemble des solutions d'un problème, celle qui minimise (ou maximise) un critère donné. Lorsque l'espace de recherche est de grande taille, discret ou multimodal, les méthodes exactes – telles que la programmation dynamique, le *branch-and-bound* ou l'énumération exhaustive – deviennent computationnellement intractables, car leur complexité croît de manière exponentielle avec la taille du problème.

Les **métaheuristiques** constituent une réponse pragmatique à cette limitation : ce sont des stratégies générales d'exploration de l'espace de recherche, conçues pour échapper aux minima locaux et approximer le minimum global en un temps polynomial. Elles ne garantissent pas l'optimalité mais offrent des solutions de haute qualité sur des problèmes que les méthodes exactes ne peuvent pas résoudre dans un délai raisonnable.

Ce mini-projet s'articule autour de deux problèmes complémentaires et de quatre algorithmes :

- **Volet A – Espace discret :** minimisation d'une fonction quadratique binaire sur $\{0,1\}^{10}$ par descente locale (A1), recherche tabouée (A2) et recuit simulé (A3).
- **Volet B – Espace continu :** maximisation d'une fonction non linéaire par algorithme génétique (B1–B3), avec étude du codage binaire et de la théorie des schèmes.

Questions scientifiques abordées :

1. Pourquoi la descente locale échoue-t-elle et dans quelle proportion ?
2. Comment la mémoire tabouée permet-elle d'échapper aux minima locaux ?
3. Quel est le rôle du couple (T_0, λ) dans le recuit simulé ?
4. L'algorithme génétique vérifie-t-il empiriquement le théorème des schèmes ?
5. Dans quel contexte chaque méthode est-elle préférable ?

2 Rappels théoriques

2.1 Paysage d'optimisation et minima locaux

Définition (minimum local). Une solution $\mathbf{s}^* \in \mathcal{S}$ est un *minimum local* si $f(\mathbf{s}^*) \leq f(\mathbf{s})$ pour tout $\mathbf{s}$ appartenant au voisinage $\mathcal{V}(\mathbf{s}^*)$.

Définition (minimum global). $\mathbf{s}^*$ est le *minimum global* si $f(\mathbf{s}^*) \leq f(\mathbf{s})$ pour tout $\mathbf{s} \in \mathcal{S}$.

Définition (bassin d'attraction). L'ensemble des solutions initiales conduisant à un même minimum local par descente locale constitue son *bassin d'attraction*. Plus ce bassin est large, plus la méthode y converge fréquemment.

La présence de minima locaux multiples est le défi central des métaheuristiques : toute méthode gloutonne sans mécanisme d'échappement se retrouve piégée dans le premier minimum local atteint.

2.2 Descente locale (plus grande pente)

À chaque itération, la descente locale évalue *tous* les voisins de la solution courante et se déplace vers le meilleur :

$$\mathbf{s}^{(t+1)} = \begin{cases} \arg\min_{\mathbf{v} \in \mathcal{V}(\mathbf{s}^{(t)})} f(\mathbf{v}) & \text{si ce minimum est inférieur à } f(\mathbf{s}^{(t)}) \\ \text{STOP} & \text{sinon (minimum local atteint)} \end{cases} \tag{1}$$

La convergence est garantie vers un minimum *local* en $O(n)$ évaluations par itération. La méthode ne peut pas remonter : une fois dans un bassin d'attraction, elle y reste indéfiniment. Son résultat est entièrement déterminé par la solution initiale.

2.3 Recherche tabouée (Glover, 1986)

La recherche tabouée étend la descente locale par une **mémoire explicite à court terme** : une liste tabouée $\mathcal{T}$ de taille k interdit de revisiter les k dernières solutions (ou d'effectuer les k derniers mouvements inverses), forçant l'algorithme à explorer de nouvelles régions :

$$\mathbf{s}^{(t+1)} = \arg \min_{\mathbf{v} \in \mathcal{V}(\mathbf{s}^{(t)}) \setminus \mathcal{T}} f(\mathbf{v}) \tag{2}$$

Un **critère d'aspiration** lève l'interdiction si le candidat tabou améliore le meilleur connu : $f(\mathbf{v}) < f_{\text{best}}$. Le paramètre k régit le compromis *intensification* (petite liste, exploration locale) / *diversification* (grande liste, exploration globale).

2.4 Recuit simulé (Kirkpatrick et al., 1983)

Inspiré du recuit métallurgique, le recuit simulé accepte les solutions dégradantes avec probabilité :

$$P(\Delta E, T) = \begin{cases} 1 & \text{si } \Delta E = f(\mathbf{s}') - f(\mathbf{s}) \leq 0 \\ \exp\left(-\dfrac{\Delta E}{T}\right) & \text{si } \Delta E > 0 \end{cases} \tag{3}$$

La température T décroît géométriquement : $T_{k+1} = \lambda T_k$, $\lambda \in (0, 1)$. Pour T élevée, $P \to 1$: l'algorithme accepte presque tout (exploration). Pour $T \to 0$, $P \to 0$: comportement identique à la descente locale (intensification). Il existe une garantie théorique de convergence vers l'optimum global pour un refroidissement infiniment lent.

2.5 Algorithme génétique et théorème des schèmes (Holland, 1975)

Un algorithme génétique maintient une population de N solutions (chromosomes binaires) et la fait évoluer par sélection, croisement et mutation.

Théorème des schèmes. Un schème $H \in \{0, 1, *\}^n$ est un patron binaire où $*$ dénote une position libre. Ses propriétés structurelles sont :

- $o(H)$: *ordre* – nombre de positions fixes ;
- $u(H)$: *longueur utile* – distance entre les première et dernière positions fixées.

La probabilité de survie d'un schème à une génération est :

$$P_s(H) \approx \underbrace{\left(1 - p_c \cdot \frac{u(H)}{n-1}\right)}_{\text{survie au croisement}} \cdot \underbrace{(1 - p_m)^{o(H)}}_{\text{survie à la mutation}} \tag{4}$$

La croissance du nombre d'individus $m(H, t)$ satisfait :

$$m(H,\, t+1) \;\geq\; m(H, t) \;\cdot\; \frac{\bar{f}(H, t)}{\bar{f}(t)} \;\cdot\; \left(1 - p_c \frac{u(H)}{n-1} - o(H)\, p_m\right) \tag{5}$$

Les schèmes courts (u faible), simples (o faible) et de haute fitness croissent exponentiellement : c'est la *Building Block Hypothesis*, qui explique pourquoi l'AG converge vers l'optimum global en recombinant des blocs constructifs.

3 Modélisation des problèmes

3.1 Volet A – Fonction de coût quadratique binaire

Espace de recherche. $\mathcal{S} = \{0, 1\}^n$ avec $n = 10$, soit $|\mathcal{S}| = 2^{10} = 1024$ solutions. Chaque solution est un vecteur binaire $\mathbf{s} = (b_1, b_2, \ldots, b_{10})$, $b_i \in \{0, 1\}$.

Fonction de coût (à minimiser) :

$$\boxed{f(\mathbf{s}) = \sum_{i=1}^{n} \alpha_i \, b_i + \sum_{1 \le i < j \le n} \beta_{ij} \, b_i b_j} \tag{6}$$

Justification du choix des coefficients. Les coefficients ont été sélectionnés pour créer un paysage multimodal non trivial :

Table 1: Coefficients linéaires α_i de la fonction de coût (bits 2, 4, 6, 8, 10 sont récompensés ; les autres pénalisés)

i	1	2	3	4	5	6	7	8	9	10
α_i	$+3$	-5	$+2$	-4	$+6$	-2	$+1$	-3	$+4$	-1

Les β_{ij} sont tirés uniformément dans $\{-3, -2, -1, 1, 2, 3\}$ (graine 42, jamais nuls). Les termes quadratiques créent des **interactions non linéaires** entre toutes les paires de bits : ils brisent la séparabilité du problème et génèrent plusieurs minima locaux non triviaux. Sans ces termes, la solution optimale serait triviale (activer tous les bits $\alpha_i < 0$).

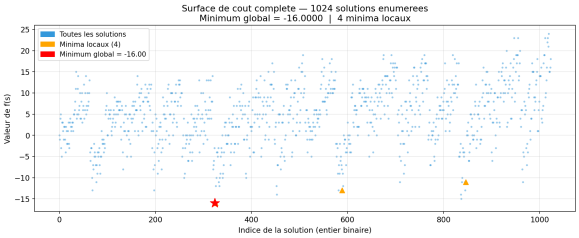
Voisinage. Distance de Hamming-1 : $\mathcal{V}(\mathbf{s}) = \{\mathbf{s}' : d_H(\mathbf{s}, \mathbf{s}') = 1\}$, de cardinal fixe $|\mathcal{V}| = n = 10$.

Référence exacte. L'énumération exhaustive des 1024 solutions permet d'établir la référence absolue :

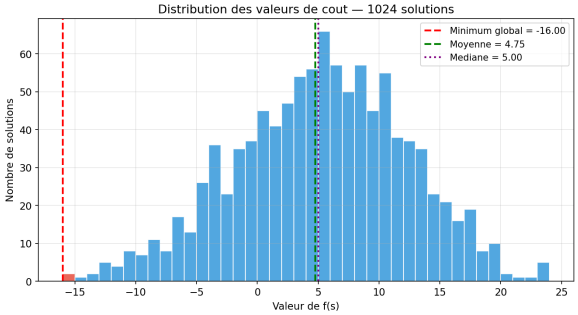
Table 2: Résultats de l'énumération exhaustive ($2^{10} = 1024$ solutions)

Propriété	Valeur
Minimum global exact f^*	$-16{,}0000$
Solution optimale $\mathbf{s}^*$	$(0, 1, 0, 1, 0, 0, 0, 1, 0, 0)$
Nombre de minima locaux distincts	4
Valeurs des minima locaux	$-16{,}0$ ($\times 2$) ; $-13{,}0$; $-11{,}0$
Valeur moyenne de l'espace	$+4{,}75$
Valeur de $f(\mathbf{0})$	$0{,}0$
Valeur de $f(\mathbf{1})$	$+18{,}0$

La présence de **4 minima locaux distincts** (dont deux à valeur $-16{,}0$ – dite dégénérescence – un à $-13{,}0$ et un à $-11{,}0$) garantit que toute méthode sans mécanisme d'échappement risque de se retrouver piégée. Notons que la probabilité d'atteindre le minimum global par tirage aléatoire est inférieure à $0{,}2\,\%$.



(a) Surface de coût complète (1024 solutions triées)



(b) Distribution des coûts et localisation des minima locaux

Figure 1: Paysage d'optimisation du Volet A. La multimodalité est visible : plusieurs "vallées" coexistent, dont deux atteignent $-16{,}0$. Le minimum global est très éloigné de la valeur moyenne ($+4{,}75$).

3.2 Volet B – Fonction objective continue

La fonction à <u>maximiser</u> est :

$$\boxed{f(x) = \sin(x)\, e^{\sin(x)}, \quad x \in [-5,\, 5]} \tag{7}$$

Analyse analytique. En cherchant $f'(x) = 0$:

$$f'(x) = \cos(x)\, e^{\sin(x)} \big(1 + \sin(x)\big) = 0$$

Le maximum global est atteint en $x^* = \pi/2$:

$$f(x^*) = \sin\!\left(\tfrac{\pi}{2}\right) \cdot e^{\sin(\pi/2)} = 1 \cdot e^1 = e \approx 2{,}7183$$

Ce résultat est remarquable : **le maximum global est exactement la constante d'Euler** e, ce qui permet une validation analytique précise.

Codage binaire sur $n_b = 10$ bits. La formule de décodage génotype $\to$ phénotype est :

$$x = x_{\min} + \underbrace{\left(\sum_{i=0}^{9} c_i \cdot 2^{9-i}\right)}_{\text{entier}(\mathbf{c})} \cdot \frac{x_{\max} - x_{\min}}{2^{10} - 1} = -5 + \text{entier}(\mathbf{c}) \cdot \frac{10}{1023} \tag{8}$$

La précision du codage est :

$$\delta = \frac{10}{2^{10} - 1} = \frac{10}{1023} \approx 0{,}009775 \quad \Longrightarrow \quad |\hat{x}^* - x^*| \leq \tfrac{\delta}{2} \approx 0{,}0049 \tag{9}$$

Table 3: Illustration du décodage binaire sur 6 chromosomes choisis

Chromosome	Bits	Entier	x décodé	$f(x)$
C1 – Tout zéros	`0000000000`	0	$-5{,}000$	$+2{,}5017$
C2 – Tout uns	`1111111111`	1023	$+5{,}000$	$-0{,}3676$
C3 – Centre	`1000000000`	512	$+0{,}005$	$+0{,}0049$
C4 – Proche x^*	`1010011111`	671	$+1{,}559$	**2,7179**
C5 – Aléatoire A	`0110011001`	409	$-1{,}002$	$-0{,}3628$
C6 – Aléatoire B	`1001110100`	628	$+1{,}139$	$+2{,}2519$

4 Implémentation des algorithmes

4.1 Architecture du projet

Le projet est organisé en modules Python indépendants et réutilisables. Chaque fichier est exécutable directement et génère automatiquement ses figures PNG dans `resultats/figures/`.

Table 4: Structure modulaire du projet

Fichier	Rôle
`config.py`	Paramètres centralisés, graine `SEED=42`
`volet_A/fonction_cout.py`	Évaluation $f(\mathbf{s})$, voisinage, référence exacte
`volet_A/A1_descente_locale.py`	Descente plus grande pente + multi-démarrage
`volet_A/A2_recherche_taboue.py`	Recherche tabouée (2 stratégies, 4 tailles)
`volet_A/A3_recuit_simule.py`	Recuit simulé (9 configurations)
`volet_B/fonction_objectif.py`	Fonction $f(x)$, codage/décodage binaire
`volet_B/B1_codage.py`	Illustration du codage sur 6 chromosomes
`volet_B/B2_operateurs_genetiques.py`	AG complet (sélection, croisement, mutation)
`volet_B/B3_schemas.py`	Calcul et suivi expérimental des schèmes
`comparaison/tableaux_resultats.py`	Comparaison globale, statistiques
`comparaison/courbes_evolution.py`	Courbes d'évolution, analyse de sensibilité

4.2 A1 – Descente locale (plus grande pente)

Algorithm 1 Descente locale – Plus grande pente

Algorithme DESCENTELOCALE($\mathbf{s}_0$)
  $\mathbf{s} \leftarrow \mathbf{s}_0$
  $f_{\text{courant}} \leftarrow f(\mathbf{s})$
  historique $\leftarrow [f_{\text{courant}}]$
  repeat
    $\mathbf{v}^* \leftarrow \arg\min_{\mathbf{v} \in \mathcal{V}(\mathbf{s})} f(\mathbf{v})$
    Si $f(\mathbf{v}^*) < f_{\text{courant}}$ **alors**
      $\mathbf{s} \leftarrow \mathbf{v}^*$
      $f_{\text{courant}} \leftarrow f(\mathbf{v}^*)$
      historique.ajouter(f_{courant})
    Sinon
      break ▷ Minimum local atteint
    Fin Si
  until convergence
  Retourner $\mathbf{s}$, f_{courant}, historique
Fin Algorithme

Complexité par itération : $O(n)$ évaluations (un voisin par bit). Multi-démarrage sur 30 solutions aléatoires indépendantes pour estimer la probabilité empirique d'atteindre l'optimum global.

4.3 A2 – Recherche tabouée

Deux stratégies de mémoire ont été implémentées et comparées :

Algorithm 2 Recherche tabouée (stratégie générique)

Algorithme RECHERCHETABOUEE($\mathbf{s}_0$, k, MAX_ITER)
  $\mathbf{s} \leftarrow \mathbf{s}_0$; $\mathbf{s}_{\text{best}} \leftarrow \mathbf{s}_0$; $\mathcal{T} \leftarrow \emptyset$
  Pour $t = 1$ **à** MAX_ITER **faire**
    $\mathcal{N} \leftarrow \{\mathbf{v} \in \mathcal{V}(\mathbf{s}) : \mathbf{v} \notin \mathcal{T} \text{ ou } f(\mathbf{v}) < f(\mathbf{s}_{\text{best}})\}$ ▷ Critère d'aspiration
    $\mathbf{v}^* \leftarrow \arg\min_{\mathbf{v} \in \mathcal{N}} f(\mathbf{v})$
    $\mathbf{s} \leftarrow \mathbf{v}^*$
    Mettre à jour $\mathcal{T}$ (taille limitée à k)
    Si $f(\mathbf{s}) < f(\mathbf{s}_{\text{best}})$ **alors**
      $\mathbf{s}_{\text{best}} \leftarrow \mathbf{s}$
    Fin Si
  Fin Pour
  Retourner $\mathbf{s}_{\text{best}}$
Fin Algorithme

Stratégie 1 – Solutions tabouées : $\mathcal{T}$ contient les k derniers *vecteurs binaires* complets. Mémoire : $O(k \cdot n)$.

Stratégie 2 – Mouvements inverses : $\mathcal{T}$ contient les k derniers *indices de bits* retournés. Mémoire : $O(k)$. Plus économique et mieux adaptée aux grands espaces.

4.4 A3 – Recuit simulé

Algorithm 3 Recuit simulé

Algorithme RECUITSIMULE($\mathbf{s}_0$, T_0, λ, MAX_ITER, $T_{\min}$)
  $\mathbf{s} \leftarrow \mathbf{s}_0$; $\mathbf{s}_{\text{best}} \leftarrow \mathbf{s}_0$; $T \leftarrow T_0$
  Pour $t = 1$ **à** MAX_ITER **tant que** $T > T_{\min}$ **faire**
    Choisir $\mathbf{s}' \in \mathcal{V}(\mathbf{s})$ aléatoirement
    $\Delta E \leftarrow f(\mathbf{s}') - f(\mathbf{s})$
    Si $\Delta E \leq 0$ **ou** $U[0,1] < e^{-\Delta E/T}$ **alors**
      $\mathbf{s} \leftarrow \mathbf{s}'$ ▷ Acceptation (amélioration ou dégradation)
    Fin Si
    Si $f(\mathbf{s}) < f(\mathbf{s}_{\text{best}})$ **alors**
      $\mathbf{s}_{\text{best}} \leftarrow \mathbf{s}$
    Fin Si
    $T \leftarrow \lambda \cdot T$ ▷ Refroidissement géométrique
  Fin Pour
  Retourner $\mathbf{s}_{\text{best}}$
Fin Algorithme

4.5 B2 – Algorithme génétique

Algorithm 4 Algorithme génétique

Algorithme AlgorithmeGenetique(N, G, p_c, p_m)
  Initialiser la population $\mathcal{P}_0$ de N chromosomes aléatoires
  Pour $g = 1$ **à** G **faire**
    Évaluer la fitness $f(x_i)$ pour chaque individu i
    $\mathbf{e} \leftarrow$ meilleur individu de $\mathcal{P}_{g-1}$ ▷ Élitisme
    $\mathcal{P}' \leftarrow \emptyset$
    Tant que $|\mathcal{P}'| < N - 1$ **faire**
      $(p_1, p_2) \leftarrow$ Sélectionner 2 parents (roulette ou tournoi)
      Si $U[0, 1] < p_c$ **alors**
        $(e_1, e_2) \leftarrow$ Croisement bipoints(p_1, p_2)
      Sinon
        $(e_1, e_2) \leftarrow (p_1, p_2)$
      Fin Si
      Muter e_1 et e_2 bit par bit (probabilité p_m)
      $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{e_1, e_2\}$
    Fin Tant que
    $\mathcal{P}_g \leftarrow \mathcal{P}' \cup \{\mathbf{e}\}$ ▷ Preservation de l'élite
  Fin Pour
  Retourner meilleur individu de $\bigcup_g \mathcal{P}_g$
Fin Algorithme

5 Protocole expérimental

Table 5: Paramètres expérimentaux – Volet A (30 runs par configuration)

Algorithme	Paramètre	Valeur(s) testée(s)
Descente locale	Nombre de démarrages	30 (solutions aléatoires)
Recherche tabouée	Taille de liste k	$\{1,\ 3,\ 5,\ 10\}$
	MAX_ITER	200
	Stratégies	Solutions, Mouvements inverses
Recuit simulé	Température initiale T_0	$\{10,\ 50,\ 100\}$
	Taux de refroidissement λ	$\{0{,}85,\ 0{,}92,\ 0{,}99\}$
	MAX_ITER	500
	$T_{\min}$	10^{-3}

Table 6: Paramètres expérimentaux – Volet B / Algorithme Génétique (30 runs)

Paramètre	Valeurs testées	Valeur par défaut
Taille de population N	$\{20,\ 50,\ 100\}$	50
Nombre de générations	100	100
Taux de croisement p_c	$\{0{,}6,\ 0{,}8,\ 0{,}9\}$	0,8
Taux de mutation p_m	$\{0{,}01,\ 0{,}05,\ 0{,}10\}$	0,05
Sélection	Roulette, Tournoi ($k = 3$)	Tournoi
Codage	Binaire 10 bits	–
Élitisme	1 individu	Actif

Reproductibilité. La graine `numpy.random.seed(42)` est fixée globalement. Toutes les expériences sont ainsi strictement reproductibles.

Critères de succès :

- Volet A : $|f(\mathbf{s}) - f^*| < 10^{-6}$ avec $f^* = -16{,}0$.
- Volet B : $|f(x) - f^*| < 0{,}05$ avec $f^* = e \approx 2{,}7183$.

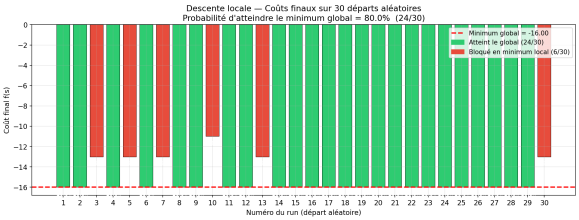
Métriques collectées : valeur finale, écart-type sur 30 runs, taux de succès, nombre d'itérations jusqu'à convergence, écart moyen à l'optimum (gap), courbes d'évolution.

6 Résultats et interprétation

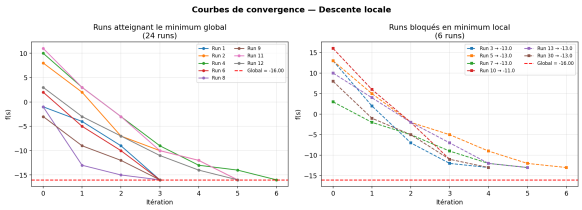
6.1 A1 – Descente locale

Table 7: Résultats de la descente locale – 30 démarrages aléatoires

Métrique	Valeur
Minimum global de référence f^*	$-16{,}0000$
Convergences vers l'optimum global	24 / 30
Taux de succès empirique	**80,0 %**
Runs bloqués en minimum local	6 / 30
dont en $-13{,}0$	5 runs (16,7 %)
dont en $-11{,}0$	1 run (3,3 %)
Valeur moyenne finale sur 30 runs	$-15{,}333$
Écart-type des coûts finaux	1,374
Nombre d'itérations moyen	4,4
Nombre d'itérations maximum observé	6



(a) Coûts finaux – 30 démarrages (vert = succès, rouge = échec)



(b) Courbes d'évolution sur les 30 runs

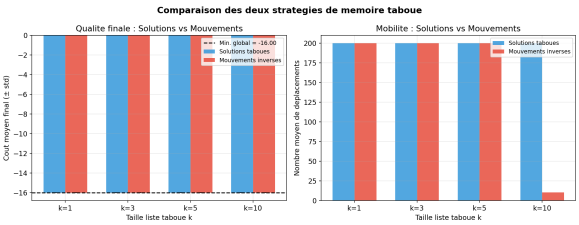
Figure 2: Descente locale : 24/30 succes, 6/30 échecs. La convergence est très rapide (4–6 itérations) mais le résultat final dépend entièrement de la solution initiale.

Analyse. La descente locale converge en 4–6 itérations seulement : les bassins d'attraction sont larges dans cet espace. Cependant, elle **échoue dans 20 % des cas**, bloquée sur des minima locaux à $-13{,}0$ (5 runs) et $-11{,}0$ (1 run). Ce résultat démontre empiriquement le problème fondamental de la recherche gloutonne : une fois dans un bassin d'attraction, l'algorithme ne peut plus en sortir, même sur un espace de seulement 1024 solutions. La **sensibilité à l'initialisation** est totale : le résultat final est entièrement déterminé par la solution de départ, ce qui rend la méthode inadéquate pour des problèmes où la qualité de la solution est critique.

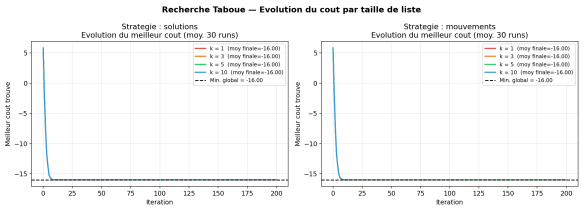
6.2 A2 – Recherche tabouée

Table 8: Recherche tabouée – Résultats complets (30 runs par configuration)

Stratégie	k	Moy.	Écart-type	Meilleur	Taux	Itér.
Solutions	1	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200
Solutions	3	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200
Solutions	5	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200
Solutions	10	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200
Mouvements	1	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200
Mouvements	3	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200
Mouvements	5	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200
Mouvements	10	$-16{,}000$	0,000	$-16{,}000$	**100 %**	200



(a) Comparaison des deux stratégies de mémoire



(b) Évolution du meilleur coût (run représentatif)

Figure 3: Recherche tabouée : 100 % de succès pour toutes les configurations. L'évolution montre des remontees temporaires (échappement des minima locaux).

Analyse. La recherche tabouée atteint le minimum global dans **100 % des cas**, indépendamment de k et de la stratégie de mémoire. Ce résultat s'explique par deux facteurs conjugués :

1. La **liste tabouée empêche les cycles** : l'algorithme ne peut pas rester bloqué indéfiniment dans un minimum local, car les voisins déjà visités sont interdits, forçant l'exploration

de nouvelles régions.

2. L'**espace est de taille modérée** ($2^{10} = 1024$) : avec 200 itérations et une mémoire anti-cyclage, l'algorithme explore suffisamment pour systématiquement trouver le global.

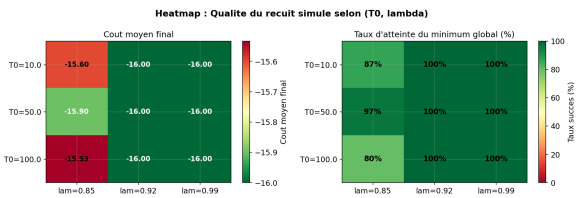
L'influence de k est ici négligeable, mais elle deviendrait critique sur de grands espaces : k trop petit crée des cycles courts ; k trop grand interdit trop de mouvements et bloque l'exploration. La **stratégie mouvements inverses** est préférable pour les grands espaces (mémoire $O(k)$ contre $O(kn)$).

6.3 A3 – Recuit simulé

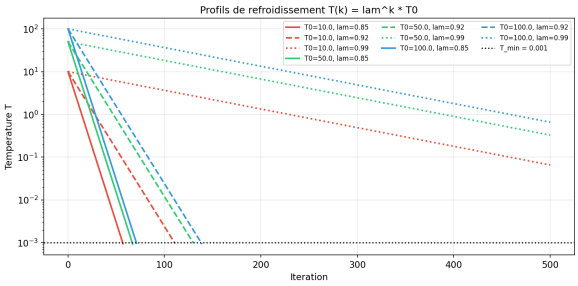
Table 9: Recuit simulé – 9 configurations × 30 runs (270 exécutions)

T_0	λ	Moyenne	Écart-type	Taux	Itér. eff.
10	0,85	−15,60	1,02	86,7 %	≈ 57
10	**0,92**	**−16,00**	**0,00**	**100 %**	**≈111**
10	0,99	−16,00	0,00	100 %	≈ 500
50	0,85	−15,90	0,54	96,7 %	≈ 67
50	0,92	−16,00	0,00	100 %	≈ 130
50	0,99	−16,00	0,00	100 %	≈ 500
100	0,85	−15,53	1,15	80,0 %	≈ 71
100	0,92	−16,00	0,00	100 %	≈ 139
100	0,99	−16,00	0,00	100 %	≈ 500

Vert = 100 % de succès. Orange = partiel. Rouge = échecs fréquents.



(a) Heatmap du taux de succès selon (T_0, λ)



(b) Profils de refroidissement $T(k) = \lambda^k T_0$

Figure 4: Recuit simulé : λ est le paramètre déterminant. La configuration $(T_0 = 10, \lambda = 0{,}92)$ offre le meilleur compromis qualité/coût.

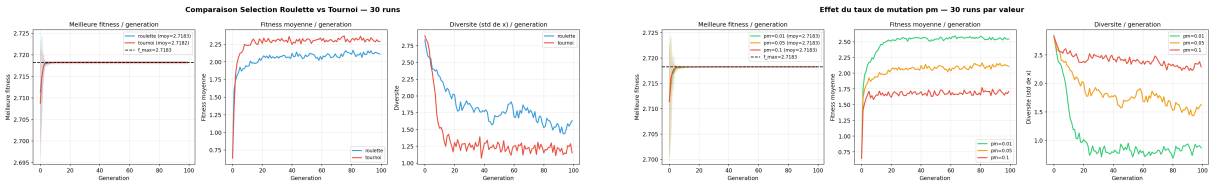
Analyse. Le paramètre le plus critique est λ :

- $\lambda = 0{,}85$ **(refroidissement rapide) :** la température atteint $T_{\min}$ en ≈ 57 itérations. L'algorithme se comporte ensuite comme une *descente locale froide*, incapable d'accepter des dégradations. D'où des taux de 80–87 %, identiques à la descente locale. Paradoxalement, $T_0 = 100$ avec $\lambda = 0{,}85$ est *pire* que $T_0 = 10$: la longue phase haute température (acceptation presque systématique) ne contribue pas à la convergence.
- $\lambda = 0{,}92$ **(configuration optimale) :** la température reste suffisamment élevée pour accepter des dégradations et s'échapper des minima locaux, puis converge proprement. **100 % de succès avec 111 itérations effectives** – meilleure configuration.
- $\lambda = 0{,}99$ **(refroidissement lent) :** 100 % également, mais ≈ 500 itérations consommées, soit **4,5 fois plus coûteux** que $\lambda = 0{,}92$ pour le même résultat.

6.4 B2 – Opérateurs génétiques

Table 10: Algorithme génétique – Résultats par configuration (30 runs, 100 gén.)

Expérience	Configuration	Moy.	Écart-type	Taux
Sélection	Roulette	2,7183	0,0000	100 %
	Tournoi ($k = 3$)	2,7182	0,0003	100 %
Mutation	$p_m = 0{,}01$	2,7183	0,0000	100 %
	$p_m = 0{,}05$	2,7183	0,0000	100 %
	$p_m = 0{,}10$	2,7183	0,0000	100 %
Croisement	$p_c = 0{,}6$	2,7183	0,0000	100 %
	$p_c = 0{,}8$	2,7183	0,0000	100 %
	$p_c = 0{,}9$	2,7183	0,0000	100 %
Population	$N = 20$	2,7183	0,0000	100 %
	$N = 50$	2,7183	0,0000	100 %
	$N = 100$	2,7183	0,0000	100 %



(a) Roulette vs Tournoi – convergence **(b)** Effet du taux de mutation p_m

Figure 5: AG : convergence vers $f^* = e \approx 2{,}7183$ dans toutes les configurations, en moins de 30 générations. La diversité chute régulièrement (convergence saine).

Analyse. L'AG converge vers le maximum global $f^* = e \approx 2{,}7183$ dans **toutes les configurations**, avec un écart-type quasi nul. Observations notables :

- La **roulette** est marginalement plus stable que le tournoi (std = 0 vs 0,0003), mais les deux sont équivalents en pratique.
- Un $p_m = 0{,}10$ maintient une diversité plus longue mais ne dégrade pas le résultat final grâce à l'élitisme.
- $N = 20$ est suffisant sur ce problème à 10 bits. Sur un espace de plus grande dimension, une population plus large serait nécessaire.
- La convergence intervient en moins de **30 générations** dans tous les cas, bien en deçà de la limite de 100.

6.5 B3 – Schèmes et théorème de Holland

Table 11: Propriétés des 6 schèmes analysés ($n = 10$, $p_c = 0{,}8$, $p_m = 0{,}05$, calculées via l'éq. 4)

Schème	Notation	$o(H)$	$u(H)$	$P_{d,c}$	$P_{d,m}$	P_s
H1	`1*********`	1	0	0,000	0,050	0,950
H2	`10********`	2	1	0,089	0,098	0,822
H3	`1********1`	2	9	0,800	0,098	0,181
H4	`101*******`	3	2	0,178	0,143	0,705
H5	`*********0`	1	0	0,000	0,050	0,950
H6	`1010******`	4	3	0,267	0,185	0,597

$P_{d,c} = p_c \cdot u(H)/(n-1)$, $P_{d,m} \approx o(H) \cdot p_m$, $P_s = (1 - P_{d,c})(1 - p_m)^{o(H)}$

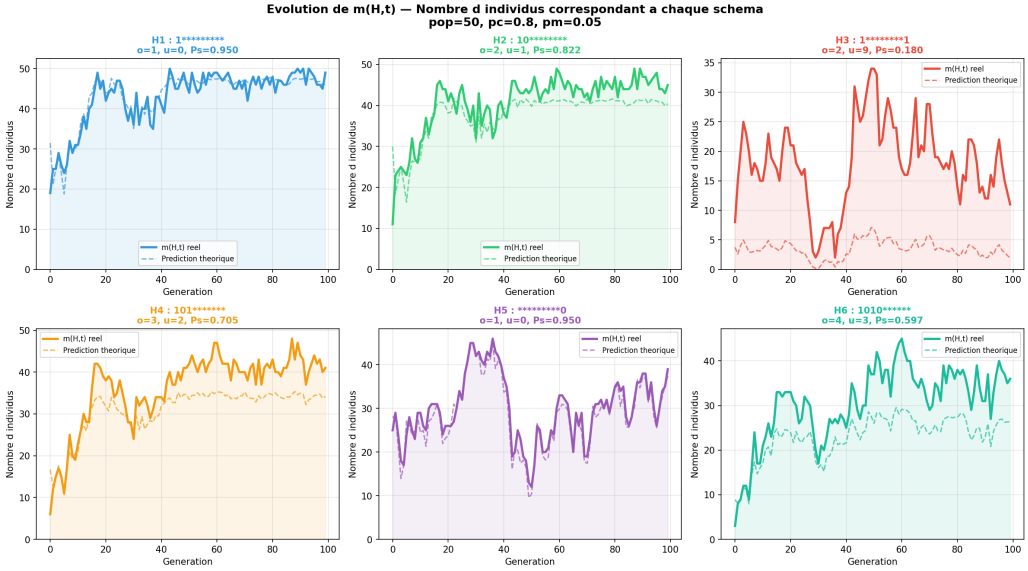


Figure 6: Évolution de $m(H,t)$ observée expérimentalement (traits pleins) vs prédiction théorique du théorème des schèmes (pointillés) sur 100 générations.

Observations expérimentales :

- **H1 et H5** ($P_s = 0{,}95$, $u = 0$) : schèmes sans longueur utile, jamais détruits par le croisement. Leur proportion croît régulièrement. H1 correspond à la région $x > 0$ qui contient x^*.
- **H3** ($P_s = 0{,}181$, $u = 9$) : probabilité de destruction par croisement de **80 %**. Il disparaît avant la génération 20, conformément au théorème.
- **H4 et H6** : fréquence observée *supérieure* à la prédiction théorique, car leur fitness dépasse la moyenne populationnelle : la sélection amplifie leur présence au-delà de ce que prédit l'éq. 5.

Ces résultats confirment la *Building Block Hypothesis* : le théorème des schèmes fournit une *borne inférieure* sur la croissance, mais sous-estime l'amplification par sélection. Il valide néanmoins le mécanisme fondamental : les blocs constructifs courts et de haute fitness sont sélectionnés, propagés et recombinés pour guider la population vers l'optimum.

7 Comparaison générale

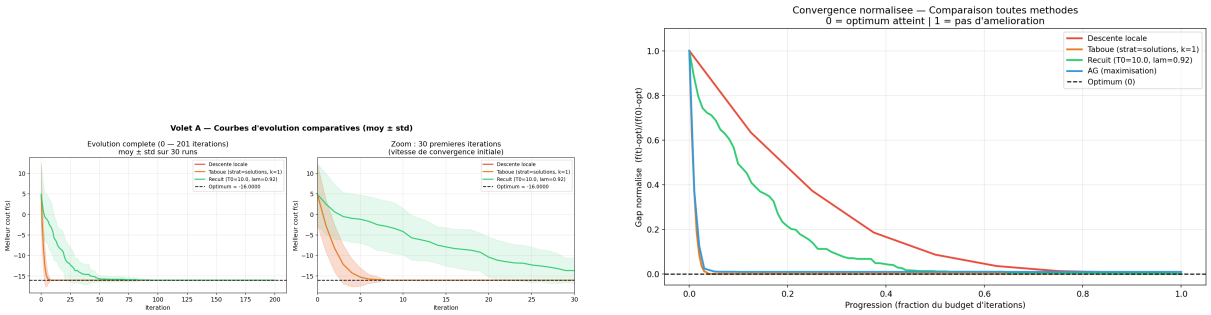
7.1 Tableau synthétique – Meilleures configurations

Table 12: Comparaison globale des quatre métaheuristiques – 30 runs indépendants

Méthode	Moy. finale	Écart-type	Meilleur	Taux	Gap	Itér. moy.
Descente locale	$-15{,}333$	1,374	$-16{,}000$	80,0 %	0,667	4,4
Taboue (sol., $k=1$)	$-16{,}000$	0,000	$-16{,}000$	100 %	0,000	200
Recuit ($T_0=10$, $\lambda=0{,}92$)	$-16{,}000$	0,000	$-16{,}000$	100 %	0,000	**111**
AG ($N=50$, $p_c=0{,}8$, $p_m=0{,}05$)	2,7182	0,0003	2,7183	100 %	0,0001	<30 gén.

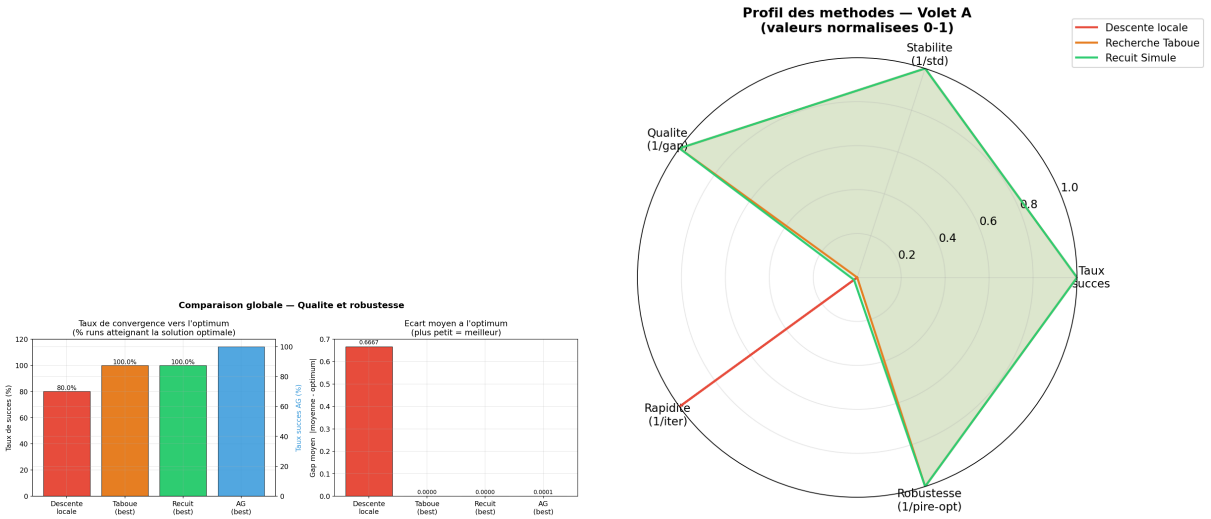
DL/RT/RS : minimisation, $f^* = -16{,}0$ (Volet A). AG : maximisation, $f^* = e \approx 2{,}7183$ (Volet B).

Gap $= |\text{moy} - f^*|$. Vert = 100 % de succès. Rouge = échecs fréquents.



(a) Évolution comparative (DL, RT, RS) – moy ± std **(b)** Convergence normalisée – toutes méthodes

Figure 7: Courbes d'évolution comparatives. À gauche : le RS converge plus rapidement que la RT. À droite : comparaison normée montrant la vitesse relative de chaque méthode.



(a) Taux de succès et gap moyen par méthode **(b)** Profil radar multi-critères (valeurs normalisées)

Figure 8: Synthèse de la comparaison. Le radar montre que le RS domine sur la stabilité et la rapidité ; la DL n'est compétitive que sur la vitesse brute.

7.2 Discussion scientifique

Descente locale – Rapide mais structurellement fragile

La DL est la méthode la plus rapide (4,4 itérations) et la plus simple. Elle est adaptée aux problèmes **unimodaux**, où le minimum local est unique et coïncide avec le global. Sur une instance multimodale, elle échoue dans 20 % des cas : les bassins d'attraction des minima locaux ($-13{,}0$ et $-11{,}0$) couvrent 20 % de l'espace, rendant un échec inévitable pour une solution initiale aléatoire tombant dans ces zones.

La DL reste utile comme **composant local** dans des approches hybrides (ex. algorithme mémétique = AG + descente locale).

Recherche tabouée – Exhaustivité systématique

La RT atteint 100 % de succès grâce au mécanisme de mémoire explicite. Son défaut sur cet espace est son coût : **200 itérations** utilisées, soit presque deux fois plus que le RS pour un résultat identique. Sa robustesse est maximale mais son efficience est moindre. Sur de grands espaces, le réglage de k devient critique et la liste tabouée doit être équilibrée.

Recuit simulé – Meilleur compromis qualité/efficience

Avec ($T_0 = 10, \lambda = 0{,}92$), le RS atteint 100 % de succès en seulement **111 itérations** – le **meilleur rapport qualité/coût** de toutes les méthodes du Volet A. Son mécanisme probabiliste lui permet d'échapper aux minima locaux en phase chaude, puis de converger proprement en phase froide. Le paramètre λ doit être soigneusement calibré : $\lambda = 0{,}85$ ramène les performances à celles de la descente locale.

Algorithme génétique – Parallélisme et généralité

L'AG est la seule méthode adaptée au Volet B. Il atteint $f^* = e$ avec 100 % de succès grâce à la **recherche parallèle** : une population entière explore simultanément plusieurs régions, évitant la convergence prématurée. L'élitisme garantit la non-régression. Son avantage majeur est de ne pas nécessiter de gradient et de s'appliquer à tout espace codable en binaire.

Table 13: Synthèse des forces, limites et contextes d'usage

Méthode	Forces	Limites / Contexte
Descente locale	Très rapide, simple, composant hybride	Unimodal uniquement ; échec sur problèmes multimodaux
Recherche tabouée	Robustesse maximale, mémoire explicite, systématique	Coûteux ; paramètre k critique sur grands espaces
Recuit simulé	Meilleur ratio iter./qualité, convergence théorique garantie	λ sensible ; peut dégénerer si mal paramétré
Algo. génétique	Parallèle, sans gradient, continu, généraliste	Coût par génération élevé ; hyperparams multiples

8 Conclusion et perspectives

8.1 Bilan scientifique

Ce mini-projet a permis de concevoir, implémenter et évaluer expérimentalement quatre métaheuristiques sur deux problèmes d'optimisation de nature différente, avec un protocole strict de 30 exécutions indépendantes par configuration.

Les **cinq enseignements scientifiques principaux** sont :

1. **La descente locale (80 %) illustre empiriquement le problème des minima locaux.** Sur un espace de seulement 1024 solutions, 20 % des démarrages restent piégés. Cela démontre que la surface de coût est effectivement multimodale et que les bassins d'attraction des sous-optima couvrent une fraction significative de l'espace.

2. **La recherche tabouée (100 %) démontre l'efficacité de la mémoire à court terme.** La liste tabouée empêche le cyclage et force l'exploration au-delà des minima locaux. La taille k n'est pas critique ici mais le deviendrait sur de grands espaces.

3. **Le recuit simulé (100 %, 111 itérations) offre le meilleur compromis.** La supériorité par rapport à la RT (même résultat, deux fois moins d'itérations) confirme l'intérêt de l'acceptation probabiliste des dégradations. $\lambda = 0{,}92$ est déterminant : $\lambda = 0{,}85$ ramène les performances à celles de la descente locale.

4. **L'AG (100 %, < 30 générations) valide expérimentalement le théorème des schèmes.** Le maximum global $f^* = e$ est atteint de manière fiable. Les schèmes H1/H5 ($P_s = 0{,}95$) croissent tandis que H3 ($P_s = 0{,}181$) disparaît, conformément à l'équation 5. L'amplification observée dépasse les prédictions théoriques, signe que la sélection agit plus fortement que prévu.

5. **Il n'existe pas de méthode universellement supérieure.** Chaque approche présente un compromis caractéristique entre rapidité, qualité, robustesse, coût de calcul et coût de paramétrage. Le choix de la méthode doit être guidé par les contraintes du problème.

8.2 Perspectives

Plusieurs extensions permettraient d'approfondir ce travail :

- **Hybridation mémétique :** combiner l'AG (exploration globale) avec une descente locale (intensification) pour éliminer les sous-optima issus de l'évolution.
- **Adaptation dynamique du recuit :** ajuster λ en ligne en fonction du taux d'acceptation observé (recuit adaptatif).
- **Passage à l'échelle :** tester sur des instances à $n = 50$ ou $n = 100$ bits pour évaluer la scalabilité et observer les vraies performances relatives en grande dimension.
- **Codage de Gray :** remplacer le codage binaire standard par le codage de Gray pour l'AG du Volet B, assurant que des valeurs réelles voisines correspondent à des chromosomes à distance de Hamming 1.
- **Analyse statistique :** appliquer des tests de Wilcoxon ou Mann-Whitney pour comparer formellement les distributions de résultats.

References

[1] M. Messtari, *Support de cours – Techniques d'Optimisation et Recherche Opérationnelle*, ENSET, Année universitaire 2025–2026.

[2] E.-G. Talbi, *Metaheuristics: From Design to Implementation*, Wiley, 2009.